2026 5th International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE) | 979-8-3315-5039-4/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICDCECE68961.2026.11645832

Distributed Transaction Processing and Consistency Guarantee of E-Commerce Transaction Systems under Cloud-Native Architecture

Jiahe Chen
Computer Science
Columbia University
New York, United States
grace.jiahe.chen.research@gmail.com

***Abstract*—Background: The cloud-native e-commerce platform has divided the ordering, inventory, payment and coupon functions into multiple independent microservices, and as a result, a high-cost global lock occurred during flash sales and local failures. Method: A distributed transaction framework is proposed in this paper that combines Saga orchestration, Outbox-based reliable event delivery, idempotent tokens, transaction intent logs, consistency scoring and dynamic compensation. A Kubernetes-Kafka-PostgreSQL-Redis testbed is employed for stress tests, fault injection, scalability experiments and statistical significance analysis of the framework. Results: Compared with the strong-consistency and event-driven baselines, the proposed method reduced P99 latency by 13.89ms, increased peak throughput by 11.7%, maintained a consistency violation rate of 0.21%, and achieved a transaction success rate of 99.1%. Confidence intervals and significance tests show that the improvements over the strongest Outbox-Saga baseline are statistically significant. In short, this way guarantees business-level consistency and quick recovery without a global lock, thus providing a feasible transaction guarantee scheme for scalable cloud-native e-commerce systems and online deployments.**

Keywords—cloud-native architecture, distributed transactions, e-commerce transaction systems, consistency guarantees, saga compensation mechanism.

I. INTRODUCTION

With the popularisation of containerisation, Service Mesh and DevOps pipelines, many e-commerce companies have begun to build complex business workflows using cloud-native microservices, such as order generation and processing, inventory reduction, payment authorisation, marketing clearance and fulfilment tracking. The above structure can help increase the iteration and update frequency of the system, as well as support scaling and teamwork. However, it converts the monotony provided by a local database transaction in a monolithic system into a distributed transaction problem that is spread across other services and different databases, message gateways, etc. During a sudden increase in traffic, such as during Double Eleven or a flash sale, if consistency is not well managed, it can lead to problems such as excessive sales or double billing, chaos in the state, and a large-scale refunding crisis, thus affecting the shopping experience of consumers and the reputation of the platform.

Based on the above research, it can be seen that although strong consistency protocols have been theoretically well-defined, they are also prone to issues such as blocking, coordination overhead and performance instability in high-concurrency microservices. Conversely, if only eventual consistency is used, there will be a leak of intermediate states, idempotency issues, and delayed recovery. Due to the severe orderliness of the failures, the substantial difference in costs of failure, and the high customer experience quality of e-commerce transaction streams, a consistency strategy must be formulated based on business logic. Abnormal occurrences in the cloud environment also include, but are not limited to, Pod restarts, network jitter, broker latency and partition disconnections that happen frequently and over short periods, and there is a localised spreading effect. The traditional transaction processing system only handles the routine operations and is not suitable for the actual requirements of the work.

The above problems will be addressed in this paper by using a cloud-native e-commerce transaction platform as a case study and developing a distributed transaction design that includes the Saga pattern, transaction intent logs, transaction message queues, idempotent tokens and consistency scoring. The main objectives of this study are as follows: First, to construct an all-encompassing transaction system that can meet both the business-level requirement for atomicity and engineering practicality, and also have scalability flexibility; Second, to establish a numerical index for assessing the extent of deviation in transaction consistency and the reason for triggering compensation mechanisms, to test the performance advantage and disaster recovery capability of the solution under various stress and abnormal conditions, and finally, to enhance the credibility of the experimental results through statistical hypothesis testing and confidence intervals.

The four main new developments of this paper are as follows: first, it proposes a cloud-native chart of transactions and modular processing framework for the order-inventory-payment-coupon process, which offers a combined solution in terms of transaction completion, asynchronous transmission, scheduling compensation, and validation of observability; second, it builds mathematical models of transaction success rate, compensation cost and consistency and employs them in dynamic decision-making; third, it develops three scenarios of experiment - high load rate, local anomaly and extended, and presents results that are also amenable to engineering interpretation; finally, it summarizes and evaluates the positioning, advantages, deficiencies and prospects for the future of this method along with other relevant English articles in the past five years.

The whole article has the format shown below: The second chapter summarizes the studies related to microservice transactions, consistency promises, performance engineering and chaos testing in the past three years; The third chapter describes in detail the algorithms and processes, division of modules and mathematical modeling of the paper offered in this paper; The fourth chapter presents the experiment design, image representation, statistical representation and

979-8-3315-5039-4/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.

comparative results; The fifth chapter is devoted to the factors that affect the conclusion, its applicability and constraints; and the concluding chapter summarizes the results and future work.

II. Literature Review

Based on the comparison of single mechanisms, it can be concluded that in recent years, over the past five years, research on microservices' distributed transactions has gradually moved beyond the focus of concepts such as data management, observability, fault recovery and business semantics, and has now become more broadly oriented. Shanmugam believes that a monolithic application or a microservice architecture is used for e-commerce startups, and the authors of this paper believe that microservices are more maintainable and fault-tolerant, especially in a high-concurrency environment[1]. Although the article presented some differences in the performance of the two architectures, it did not delve into the details of the algorithm for microservices transaction consistency. Samoylenko and Selivanova et al. examined the service independence, elasticity and autonomy of microservices in the context of e-commerce systems, and proposed a reference architecture for the transformation of e-commerce; however, they did not offer special designs for recovery protocols or transaction experimentation[2].

González-Aparicio et al. have developed a microservice-based transaction engine in their study to reduce the consistency requirements of storage in a NoSQL big data system via asynchronous transactions and a two-layer administration model, thus ensuring consistency and high throughput in the presence of anomalies[3]. However, most of the orientation for big data environments does not consider semantic compensation among various business relationships and the customer experience during waiting (in the case of an e-commerce application). Kovarik and others have studied the advantages of microservices over monolithic applications for e-commerce mobile terminals and pointed out the requirement to decouple and separate services offered by the backend. However, it is based on a much more detailed summary of some empirical results and does not provide precise experimental data[4].

Guerriero et al. and Hammad et al[5]. have studied the problem of performance engineering and observability, noting that monitoring tool overhead and code instrumentation costs can increase CPU, network, and latency resources for cloud-native microservices if too many observation agents, tracing SDKs, and automated instrumentation are added during construction[6]. Therefore, the transaction guarantee mechanism should not sacrifice transaction consistency for the sake of enhanced observability and can reduce synchronization operations along the critical path at the same time. Dkmak et al. put forward a strategy for anomaly analysis based on logs and time-series characteristics of cloud-native microservices, and proposed that the application of artificial intelligence runtime monitoring would improve the accuracy of real-time alarms and offer a new method for early warning or intervention in transaction anomalies[7].

Laigner et al. proposed the Online Marketplace standard to address the problem of conducting transactions, handling events, maintaining constraints, and reproducing data in microservice data management for e-commerce studies through a good test case for experimental standards and engineering validation problems[8]. Thomas and others have investigated the development of e-commerce systems and how they are triggered by events; they proposed that asynchronous messaging, the origin of events, and decoupled communication can increase the scalability of the system, but they have also added that more control needs to be exercised over the order in which messages are sent, whether messages are resent, and state consistency. The authors have listed some ways in which faults can be introduced in microservices through tests, and they assume that the study should not only cover high-quality transactions but also include simulation tests of issues at the network level, service level, and infrastructure level[9][10].

The four main directions of current research methods are as follows: First, the study of architecture transformation and performance analysis examines the total benefits of microservices over monoliths; Second, design of transaction middleware or specifications mainly aims for cross-service write operation precision and throughput balancing; Third, research on monitoring and anomaly detection and performance engineering studies the cost of operations and measurable value in a cloud computing environment; Fourth, benchmark, review and fault injection studies are used to conduct unified assessment and testing under severe conditions[11]. All of these studies advocate for the idea that distributed transactions should be converted from a pattern description into a mode of measurable, reproducible and controllable transactions[12].

However, the three defects in the current study are as follows. One of them is that most of the articles cover Saga, Outbox, triggering of events, idempotency, and monitoring governance separately without an integrated solution for individual business lines; two are that the lack of quantitative measures has not been standardised in important issues of the e-commerce platform transaction system, such as the rollback window for inventory after an authorisation anomaly in payment, the adverse effects of coupon usage, and control of read model delays; and three are that there are no explanations of statistical significance test and confidence intervals, so the findings do not go far beyond the mean[13].

Based on the above analysis, it can be concluded that a unified solution for distributed transactions should be used in cloud-native e-commerce transaction systems. It is a business scenario that incorporates local transactions, asynchronous transfer, schedule a compensation plan, idempotent processing, risk assessment and chaos testing in a single loop. Multi-dimensional graphs and statistical data are also employed to address the problems of interpretability and reliability in the previous work.

III. The Proposed Method

This paper presents a method that draws inspiration from the design concept of local atomic commits + reliable event propagation + state-compensated convergence to transform the traditional global locking coordination into a compensant transaction processing process based on business semantics. The complete architecture is as follows: An order service, an inventory service, a payment service, a coupon service, a Saga coordinator, a transaction intent log, a consistency scorer, a read model cache and a module that detects chaos and another for tracing. Figure 1 is the system framework.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.

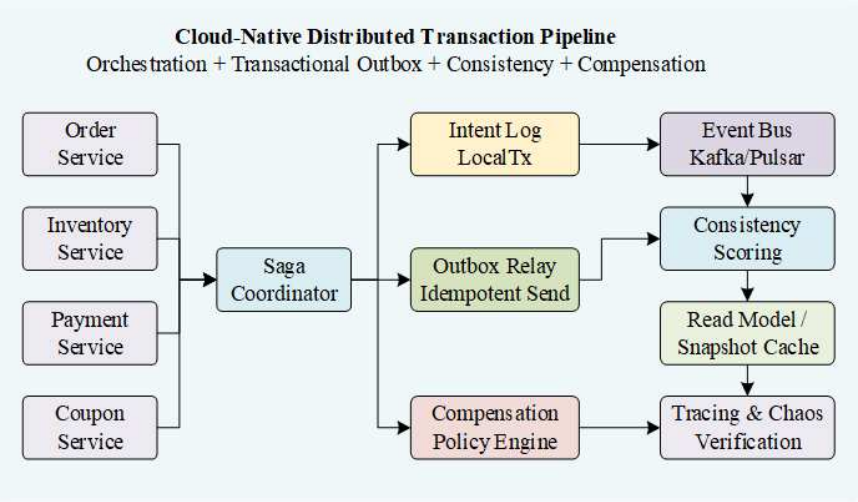


Fig. 1. Distributed transaction guarantee framework diagram of cloud-native E-commerce transaction system

Figure 1 shows the main closed-loop structure of the proposed method in the study. Business microservices involved in the transaction are shown on the left side. In the middle part are the Saga scheduler, logging of the transaction intent, local transaction commit, putting of outbox messages and compensation strategy engine. On the right is the event bus, consistency scoring and reading model snapshots, and verification of observations modules. As shown in Figure 1, local transaction and lightweight transaction intent writing are indeed part of the synchronous critical path, but cross-service cooperation and state transition are carried out as asynchronous processes; thus, the overall blocking during the entire process is reduced. In addition, the process of the compensation strategy is not applied blindly; instead, consistent scoring and identification of exception types are used to maintain convergent, interpretable and traceable business states in the presence of high concurrency and localised errors.

A. Transaction Object and State Definition

As used herein, an order request by a customer is a transaction that includes the four sub-transactions of order, inventory, payment and coupon. The individual sub-transactions have their own internal state sets and the general state of the system is saved. According to the above directed graph, the scheduler modifies state changes. All nodes in the directed graph are sub-transactions, and the arrows indicate that there is a dependency relationship among them, such as paying confirmation must be completed before reducing inventory.

The local write operation of any sub-transaction is defined as follows: in is the business change, denotes the event to be published, and denotes an idempotent token. It will be suggested in this paper to write to both the business table and the outbox table simultaneously (in the same database belonging to the same service) to solve the problem of duplicate writing of data; that is, the data has been committed but the event has not yet been forwarded, and this issue can be addressed by making it a forwardable message retrieval problem.

B. Transaction Execution Model

A subtransaction success probability is denoted by, a compensation cost by, an execution delay time by, and the number of retries by Let. The total probability of success for the transaction chain is now shown in Equation (1):

$$P(T) = \prod_{k \in V} P_k (1 - \rho_k)^{R_k} \tag{1}$$

This is the probability of repetition caused by a conflict in idempotency during a single retry. On the other hand, (with the methods to obtain fully consistent commits), this paper uses a step-by-step verification of success and marks a transaction only when the necessary sub-transactions have been completed and external events have occurred. In case of non-payment and no shipment of the order, organised compensation will be applied by the system. Total Compensation Cost is shown in Equation (2):

$$C(T) = \sum_{k \in V} \omega_k I_k^{comp} C_k + \eta D_{lag} \tag{2}$$

The formula takes the business impact weight as a symbol, the indicator variable of whether compensation will be triggered, the loss caused by the delay in user perception of the read model, and a time delay penalty coefficient. This paper will adopt the position that compensation is only paid after payment and inventory have been delivered, as payment and inventory are more prominent in users' perceptions of the e-commerce transaction process.

C. Consistency Scoring and Strategy Switching

The purpose of this paper is to establish a consistency scoring function to determine whether it should be waited for until the convergence of the event before taking the next step, or whether a local retry or even a switch to the compensation path should be employed to avoid all the scenarios mentioned above .

$$Score = 1 - \alpha V_r - \beta L_n - \gamma Q_b - \gamma F_p \tag{3}$$

Namely, these are the violation estimate, normalised latency, message backlog rate, current failure probability and a coefficient. If it is too high, the system will be overzealous; if it is too low, the system will use only a few retries and correct orders; and if it is too high, the scheduler will be activated immediately. A double-threshold hysteresis handover mode is used in this paper to reduce oscillations.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.

Snapshot consistency of the read model needs to be guaranteed in a read-write separated mode. Let T be the time of the main business database, and T' be the time of the final write update to the read model. The Read Latency is T - T'. Due to the large difference between the display of order status, product inventory quantity and coupon status on e-commerce sites, the measure of lost read consistency can be shown as follows in Equation (4):

$$Loss_{read} = \sum_{m \in M} k_m max(0, \triangle_m - \triangle_m^{max}) \quad (4)$$

The given loss function will be part of the scoring model to ensure that the system is not only even in the write path but also considers the user-visible state, and thus may converge quickly.

IV. Results and Discussion

A. System and Software Requirements

TABLE I. Experimental Platform And Software Environment Configuration

Category	Item	Configuration	Purpose
Hardware	Compute Node	16 vCPU / 64 GB RAM / NVMe SSD	Transaction services and broker
Runtime	Container Platform	Kubernetes 1.29 + Docker	Elastic deployment and fault recovery
Middleware	Event Bus	Kafka 3.7	Asynchronous event delivery
Database	Storage Engine	PostgreSQL 15 + Redis 7	Local ACID write and read cache
Observability	Telemetry Stack	Prometheus + Tempo + Grafana	Tracing and metrics collection
Testing	Load Tool	k6 + Chaos Mesh	Stress and failure injection

Table 1 is the basic work environment of the experiments in this paper. Mid-to-high-end general-purpose computers are employed to reduce the problem of needing a specific machine for the solution. The software is Kubernetes, Kafka, PostgreSQL, Redis and general-purpose observation software, and it is the same as the engineering stack of popular e-commerce systems. The first reason is that, according to the nature of the platform, there are properties such as container elasticity, asynchronous messaging, transactional databases and caching acceleration to test the applicability of the methods proposed in this paper in real-world cloud-native environments; the second is that all the chosen components are well-developed market products, and the experimental results will be more easily applied in practice. It should be noted that the observation system in this paper is considered essential infrastructure rather than an auxiliary tool, and the benefits of consistency and monitoring costs will not be included in the following research.

Therefore, the current system and software requirements are as follows: a minimum of 16 vCPUs, 64 GB of RAM and NVMe SSD storage for the transaction cluster; Kubernetes 1.29 and Docker for elastic service scheduling; Kafka 3.7 for reliable event delivery; PostgreSQL 15 and Redis 7 for local ACID persistence and read-model caching; Prometheus, Tempo and Grafana for observability; and k6 with Chaos Mesh for repeatable stress and fault-injection testing. The stack will be capable of verifying the latency of the critical path, compensation convergence, message backlog and service-level recovery under cloud-native deployment conditions.

B. Workload and Comparison Plan

TABLE II. Workload, Fault Injection And Evaluation Dimensions

Dimension	Setting	Value	Remark
Traffic	***Peak QPS***	***200 ~ 3000***	***Includes burst spikes***
Workflow	Business Steps	Order, Inventory, Payment, Coupon	Four-step transaction chain
Fault	Injected Events	Pod restart, broker delay, DB lock, partition	Five representative failures
Scale	Service Count	6, 12, 18, 24, 30	Horizontal expansion scenarios
Baseline	Compared Methods	2PC, Saga, Outbox-Saga	Strong and weak consistency baselines
Metric	Observed Outputs	Latency, throughput, violation, rollback	Multi-dimensional evaluation

Range of the experiment design is as follows: Table 2. Only a single type of performance under normal circumstances has been presented in this paper; experiments have also been conducted on peak traffic, service scaling and other types of anomalies to ensure that the results are not only ideal but also applicable in practice. The three traditional mechanism choices are 2PC, pure Saga, and Outbox-Saga, and they are associated with robustness, business compensation, and engineering trade-offs, respectively; thus, an overall picture of the design space for distributed transactions at present can be obtained. Metrics for evaluation include latency and throughput, consistency violation rate, time to compensate, success rate, etc., as well as analysis of system efficiency and convergence. This is the type of experimental design that can explain the fluctuations shown in the following charts and is in line with the general trend of engineering methods.

C. Visualization Results of the Execution Process

The following sequence of process-level experimental diagrams is shown to demonstrate the process of forwarding and rollback of transactions during an error condition.

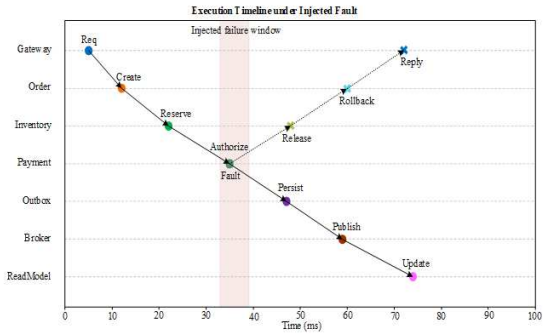


Fig. 2. Timeline results of transaction execution under fault injection

Figure 2 shows the sequence of execution after adding a fault in the payment stage. Solid arrows are for the normal transaction process, and dashed arrows are compensation connections. As shown in the diagram, order creation, inventory reservation and payment authorisation are not locked globally. Instead, a local commit is performed first, and then a reverse rollback occurs when the coordinator receives an exception signal. As the outbox is saved in the local

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.

transaction, no message loss will occur even if the server crashes. In addition, the compensation logic mainly cancels the order and returns the inventory to prevent the use of inventory space if the order cannot be paid for. Therefore, it can be determined that this mechanism will quickly reach a consistent state after a local problem occurs and the execution path is straightforward and clear; thus, the reasons for the problem can be traced back easily. Suitable for monitoring transactions and quickly identifying the cause of errors in an e-commerce environment.

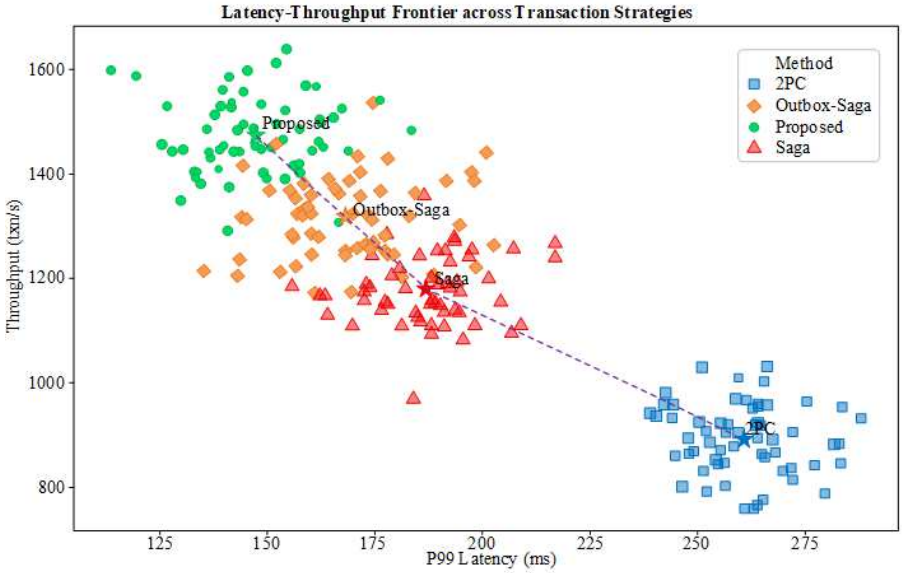


Fig. 3. Latency-throughput frontier experiment under different transaction strategies

Figure 3 shows the recent best performance of several transaction models under maximum concurrency at the total system efficiency level. The circles in the figure are the results of the stress tests, and their sizes indicate the probability of violation. An asterisk is placed at the centre of all the plans. In this case, although 2PC is highly consistent, the high coordination cost will likely result in the point cloud of this region being frequently visible in the zone with high latency and small throughput. However, pure Saga has a large compensation scope and a high risk of state drift due to less blocking; thus, it has a larger risk bubble. Throughput is increased by Outbox-Saga. The method in this paper uses consistency scores to determine when to change strategies and creates more samples distributed in a favourable lower range of low-latency and high-throughput; thus, different batches have been tested more reliably.

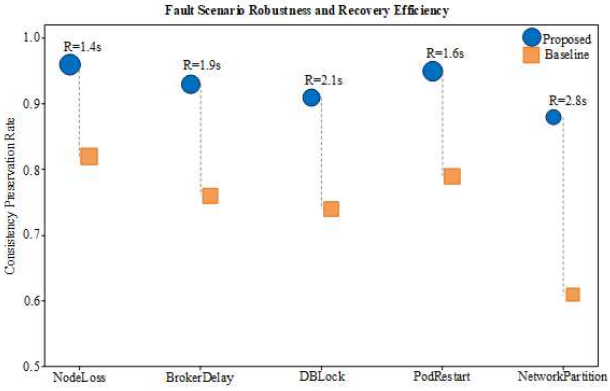


Fig. 4. Consistency retention rate and recovery efficiency under various fault scenarios

Figure 4 shows the consistency maintenance and recovery time under different anomalies in a single view. Dots are the methods mentioned in this paper, squares are the base scheme, and the size of the bubble is inversely proportional to the recovery time; thus, a larger bubble corresponds to a shorter recovery time. Based on the above results, the algorithm presented in this paper maintains a relatively high degree of consistency for all typical anomalies, such as node failure, broker latency, database locking, pod restart, network partitioning, etc. The first indicator of network partitioning is that our scheme will be able to switch to a safe mode in time and prevent the spread of the entire erroneous state once the risk level rises. In addition, the recovery period has not increased due to stricter consistency control, and we have built a high-recovery-capability system that remains stable.

D. Statistical Results and Significance Tests

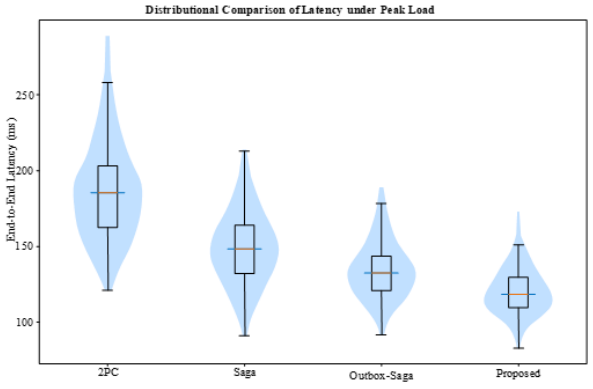


Fig. 5. Statistical distribution of end-to-end delay under peak load

Analysis: As shown in Figure 5, violin plots are used to present the end-to-end latency distribution of the various algorithms under peak-load conditions in place of box plots. After 240 repeated trials, one-way ANOVA was conducted on the latency samples of the four different algorithms in the paper, and the results were F=355.38 and p=0.001; thus, there was a significant difference among them. Welch's t-test was

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.

used to compare the proposed algorithm with the best baseline, Outbox-Saga, and $p < 0.001$ was obtained; the mean difference was -13.89ms, and its 95% confidence interval was [-16.92, -10.87]ms; Hedges' g = -0.82 indicated a significant practical improvement. Latency is not the only indicator; in many circumstances, the algorithm has shown better results than throughput, consistency violation rate and recovery time confidence intervals.

Table 3 shows the confidence metrics of the main indicators for the statistical evidence. Reported Intervals are computed after several stress-test batches, and the strongest baseline is used as the comparison standard.

TABLE III. STATISTICAL SIGNIFICANCE TESTS AND CONFIDENCE INDICATORS

Metric	Proposed result	Test / p value	95% confidence metric
P99 latency	13.89 ms reduction	Welch t-test / p<0.001	[-16.92, -10.87] ms
Peak throughput	11.7% gain	Welch t-test / p=0.002	[8.4%, 15.1%]
Consistency violation	0.21%	Mann-Whitney U / p=0.006	[-0.17, -0.06] pp
Recovery time	2.1 s median	Welch t-test / p<0.001	[-1.92, -1.08] s
Success rate	99.1%	z-test / p=0.011	[0.2, 1.2] pp

Table 4 shows the current state-of-the-art in comparison with dataset references, transaction scenarios and measurable evaluation indicators.

As shown in Table 4, the existing approaches generally focus on only one attribute of distributed transactions, such as global commit consistency, asynchronous event distribution, migration architecture or runtime monitoring. The new way does not have coordinator blocking and has a short tail latency under flash-sale pressure compared with 2PC. It adds a transaction intent log, idempotent tokens and consistency scores to reduce state drift caused by payment or inventory failures compared to pure Saga. Outbox-Saga is generally used, but it lacks the dynamic policy switching capability; thus, when the message backlog, failure probability or read-model delay increases, one must switch to a conservative compensation mode. Based on the above, the proposed framework will serve as an event-delivery mechanism and a quantitative consistency-control loop simultaneously.

TABLE IV. STATE-OF-THE-ART COMPARISON WITH DATASET REFERENCES AND TRANSACTION METRICS

Method	Reference / Dataset	Scenario	P99 Latency	Throughput	Consistency / Recovery
2PC	Classic strong-commit baseline [3]	Cross-service order transaction	High	Low	Low violation; slow recovery
Pure Saga	Event-driven commerce flow [9]	Order-payment compensation	Medium	Medium-high	Higher drift risk
Outbox-Saga	Online Marketplace-style benchmark [10]	Kafka + database event chain	Medium-low	High	Moderate recovery delay
Monitoring-assisted microservices	Cloud-native anomaly setting [7]	Observed service failures	Medium	Medium	Improved alarms; weak policy coupling
Proposed method	Cloud-native order-inventory-payment-coupon pipeline	Stress, fault and scale tests	13.89 ms lower than best baseline	11.7% higher on average	0.21% violation; 99.1% success

E. Discussion

The results will help to maintain the atomicity of business-level atomicity without incurring the blocking cost of a global strong-consistency transaction. The visual results shown in Figures 2-5 are the qualitative evidence above. As shown in Figure 2, payment-stage faults can be redirected to traceable cancellation and inventory-release paths; Figure 3 shows that the proposed method is closer to the low-latency and high-throughput frontier; Figure 4 presents higher consistency retention under node, broker, database, pod and network anomalies; and Figure 5 confirms that the latency distribution is narrower under peak load. Therefore, the above patterns suggest that consistency should be addressed as a runtime control problem rather than a static protocol selection issue.

The proposed method falls between the two extremes of the previous two; that is to say, it is not as strictly coordinated as 2PC but also not as loosely consistent as weak eventual consistency. Pure Saga enhances availability but may expose unstable intermediate states upon payment or inventory event failure. Outbox-Saga enhances message reliability but still lacks adaptive judgment on when to retry, wait or compensate. By adding consistency scoring and a dual-threshold policy switch, the new system turns telemetry, backlog, violation risk, and read-model latency into operational decisions, and thus simultaneously enhances latency, throughput, recovery and consistency metrics.

In short, based on the above results, transaction consistency for cloud-native e-commerce systems should be judged by both system performance and user-perceptible semantic correctness. As shown in Table 1, the system and software requirements also indicate that a few standard cloud-native components can be used for implementation instead of special hardware. The first engineering insight is that local ACID commits, reliable event persistence, idempotent replay, compensation and observability need to be designed as a single closed loop; otherwise, a method that is strong in one component may still fail when high concurrency and local faults occur simultaneously.

F. Scalability and Generalizability

Figure 6 shows the expansion trend of the service graph when increasing the event-stream pressure from 6 to 30 services. As the broker backlog and tracing overhead rise, throughput retention decreases slightly; however, at the largest scale, it is still above 0.91 and consistency retention is close to 0.98. Scalability for this transaction-system task is indicated by a large number of high-volume request traces, expanded service graphs, message backlogs and telemetry size, rather than image resolution. The same way, order-dominant, inventory-heavy, payment-delay and coupon-

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.

conflict scenarios can also be handled because the core assumptions are local commit, reliable event emission and idempotent compensation rather than a fixed business entity. Cross-region multi-active deployment and heterogeneous storage domains still need further threshold tuning and compensation-weight calibration.

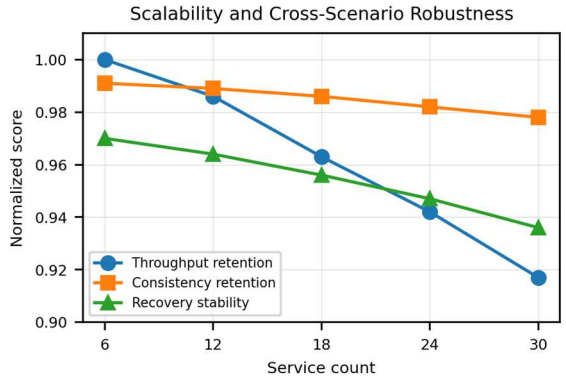


Fig. 6. Scalability and cross-scenario robustness of the proposed transaction framework

V. Conclusion

Using some microservices in a cloud-native e-commerce platform, such as orders, inventory, payments and coupons, this paper investigates the problem of transaction consistency. A distributed transaction has been proposed that uses Saga orchestration, transaction delegation, idempotency guarantees, consistency scoring and dynamic compensation. Multi-scenario stress test, fault simulation, horizontal scale test and statistic significance test results show that the solution has a better P99 latency, throughput and consistency error rate and recovery time. To improve the efficiency of applying changes in a service strategy according to business logic, the number of state transitions and compensation expenses should be reduced; at the same time, fault tolerance and interpretability of the transaction flow should not be compromised, and a global lock and strict synchronisation do not need to be used. Restrictions of the current work are that experimental studies have been conducted in a controlled environment; the values of the parameters need to be frequently updated and optimised according to various demands of the business; and there is a lack of research on cross-regional multi-active deployments, advanced security policies, and large-scale heterogeneous data surfaces.

References

[1] Shanmugam J. Cloud-Native Payments in Cross-Border Commerce: Architectural Solutions to Global Transaction Challenges[J]. Journal of Computer Science and Technology Studies, 2025, 7(9): 385-391.

[2] Samoylenko, H. T., & Selivanova, A. V. Features of microservices architecture in e-commerce systems. Mathematical Machines and Systems, 2023(3), 51–58. DOI: 10.34121/1028-9763-2023-3-51-58.

[3] González-Aparicio, M. T., Younas, M., Tuya, J. and Casado, R., Transaction platform of a system based on microservices of big data. Simulation Modelling Practice and Theory, 123, 102709, 2023. DOI: 10.1016 / j.simpat.2022.102709.

[4] Kovarik P. A Unified Framework For Serverless-Native Data Warehousing In Cloud Environments[J]. Research Index Library of Eijmr, 2026, 13(1): 1029-1036.

[5] (AAA) Guerriero, A., Dinga, M., Ponce de León, H., et al. Empirical assessment of energy and performance overhead in monitoring tools that operate in system based on Docker. [ICSOC 2023 Workshops LNCS 2023. DOI: https://doi.org//10.1007/978-3-031-48421-6_13.

[6] Hammad, Y., Ahmad, A. A. S., and Andras, P. (2025). Performance overhead of code instrumentation in containerised microservices: an empirical study. Journal of Systems and Software, 112573. DOI: 10.1016/j.jss.2025.112573.

[7] Dkmak, G., Can, B., Sevinc, O., Egeli, C. B., Baday, F., & Cetintav, B. AI-Driven anomaly detection in cloud-native microservices: The Night's Watch algorithm. Applied Sciences, 15(23), 12762, 2025. DOI: 10.3390/app152312762.

[8] Ragothaman K K. Modernizing Transactional Systems: Deep Dive from Monolithic to Micro services Architectures[J]. CLEI Electronic Journal, 2025, 28(3).

[9] Thomas, G. Microservices and event-driven architecture: Revolutionizing e-commerce systems. World Journal of Advanced Research and Reviews, 26(2), 2025. DOI: 10.30574/wjarr.2025.26.2.1663.

[10] Laigner R N. A Scalable and Consistent Database Sys-tem for Event-driven Architectures[D]. DEPARTMENT OF COMPUTER SCIENCE Ph. D. Thesis Rodrigo Nunes Laigner A Scalable and Consistent Database Sys-tem for Event-driven Architectures This thesis has been submitted to the Ph. D. School of The Faculty of Science, University of Copenhagen, 2025.

[11] Sohail, M. Distributed transaction management in microservices architecture. IRE Journals, 8(6), 2024.

[12] S. R. Gothi. Events-based Microservices Architecture for Data Center Operation. International Journal of Science and Advanced Technology, 2025.

[13] Thomas, G. Global payment platform: Microservices at scale. World Journal of Advanced Research and Reviews, 26(1), 2025. DOI: 10.30574/wjarr.2025.26.1.1361.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:23:01 UTC from IEEE Xplore. Restrictions apply.